

MASFE: An Onboard Multi-Algorithm Scheduling and Fusion Engine

For Regenerative Agriculture SmallSat Missions

Emil Wes Lambert

BSc Aerospace Engineering, TU Delft

Solo submission; NASA challenge eligibility: U.S. citizen

1 Introduction

MASFE (Multi-Algorithm Scheduling and Fusion Engine) answers NASA’s Space-to-Soil challenge by moving land-observation orchestration onboard instead of treating the spacecraft as a passive camera [1]. The target use case is regenerative agriculture, where disease and irrigation stress must be detected early enough to enable patch-level intervention instead of blanket chemical application. Current land products are typically too slow or too coarse for that workflow, even when the underlying algorithms are scientifically strong [4, 2, 3].

The proposed contribution is a two-stage, MDP-formulated scheduling policy executed as a deterministic, flight-verifiable rule set on a LEON4FT processor. A low-cost spectral screen runs every pass; a more expensive thermal confirmation step runs only when the belief state indicates likely stress. The payload is framed as a hosted module on Loft Orbital’s YAM platform and hands alerts to existing agricultural software through OpenET-linked ground services [5, 6]. The rest of the paper focuses on the judge-facing questions of creativity, feasibility, impact, business viability, and next-step readiness.

2 Proposed Solution Detail

MASFE uses a two-stage adaptive sensing loop. Stage 1 executes an EVI anomaly screen derived from MOD13A1 logic on every observed field patch. When a patch departs from its stored healthy baseline, Stage 2 runs a thermal retrieval derived from MOD11A1 and fuses both signals into a Crop Stress Composite (CSC). Only confirmed anomalous tiles are buffered for priority downlink. This makes sensing content-driven rather than timeline-driven and distinguishes MASFE from ASPEN/CASPER-style planners, which optimize scheduled activities rather than per-patch stress belief [7].

$$\text{CSC} = 0.55 \cdot \text{clip}\left(\frac{\Delta\text{EVI}}{\sigma_e}, 0, 1\right) + 0.45 \cdot \text{clip}\left(\frac{\Delta\text{LST}}{\sigma_l}, 0, 1\right) \quad (1)$$

Here $\Delta\text{EVI} = \max(0, \text{EVI}_{\text{base}} - \text{EVI}_{\text{obs}})$ and $\Delta\text{LST} = \max(0, \text{LST}_{\text{obs}} - \text{LST}_{\text{base}})$. The implementation is intentionally deterministic even though the policy is MDP-formulated: that choice favors flight qualification and traceable verification on LEON4FT hardware over a learned black-box controller.

Table 1: Mission requirements and demonstrated MASFE results.

Requirement	MASFE response
Alert latency ≤ 90 min	One-pass onboard screening, fusion, and alerting pipeline with X-band priority downlink.
Spatial resolution ≤ 30 m GSD	Simera xScale100-class multispectral imager; MODIS-derived EVI/LST logic executed at sensor-native resolution rather than MODIS ground-product resolution.
False-positive rate $\leq 15\%$	1.0% in a 120-pass simulation with 38 cloud-obscured passes, injected measurement noise, and benign stress confounders.
SmallSat SWAP compliance	9.6 W peak, 7.6 W duty-cycled average, 92.5% LEON4FT utilization, and 86% minimum battery state of charge.
Downlink efficiency	97.8% reduction versus raw downlink baseline while retaining 100% disease-event recall.

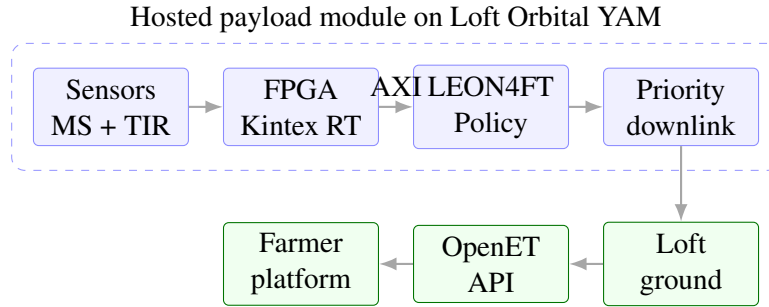


Figure 1: MASFE end-to-end architecture from onboard sensing to farmer alert delivery.

The FPGA performs radiometric preprocessing and per-pixel CSC computation, then transfers frames to the LEON4FT over AXI4-Stream in under 1 ms. The Kintex UltraScale+ RT component is rated to 100 krad total ionizing dose, which supports low-Earth-orbit deployment margin. MASFE is framed as a hosted payload module within a Loft Orbital YAM bus rather than as a full standalone 6U satellite; the SWAP budget therefore describes the payload module itself, not the host spacecraft.

Operational degradation is explicit. Above 35% battery state of charge, MASFE runs the full two-stage policy. Between 15% and 35%, it falls back to Stage 1 screening only. Below 15%, it preserves energy for stored alert downlink. This satisfies the challenge requirement to degrade gracefully under onboard resource stress [1].

3 Potential Impact

MASFE builds directly on existing NASA land-observation algorithms instead of inventing a new crop model from scratch. The novelty is orchestration: MOD13A1-style EVI screening is cheap enough to run every pass, while MOD11A1-style temperature confirmation is reserved for suspicious patches [2, 3]. That scheduling choice matters because regenerative agriculture decisions are time-sensitive. Dr. Katie Gold’s disease-detection work emphasizes that spectral stress often appears before visible symptoms and before whole-field treatment would be agronomically or economically rational [4].

In simulation, MASFE was tested over 120 passes, 50 field patches, 7 disease events, and 38 cloud-obscured passes. Relative to raw downlink, it reduced transmitted data by 97.8% and active payload energy by 22.0%. Relative to a fixed onboard pipeline, it still reduced downlink volume by 4.9% because it

avoids running or transmitting the expensive path when nothing important is happening. The more important metric is mission usefulness: disease-event recall stayed at 100%, alert precision was 99.0%, and the false-positive rate remained at 1.0%.

The farm-scale value proposition is concrete. For a 500 ha regenerative tomato operation, patch-only treatment after a MASFE alert is materially cheaper than blanket preventative spraying and can avoid both unnecessary chemical cost and avoidable yield loss. The same architecture also supports regenerative practices beyond pathogen detection: cover-crop phenology monitoring, soil-carbon MRV support, and no-till verification all benefit from adaptive revisit and selective downlink instead of fixed-rate collection.

The main limitations are honest rather than hidden. The current validation is still TRL 3 and uses synthetic proxy data instead of flight imagery. Cloud screening must be tuned on real scenes, and tropical crop calendars would require policy recalibration. Those gaps are real, but they are the right gaps for a finalist-stage concept: the system question has been answered, and the next work is hardware and data validation rather than rethinking the architecture.

4 Team Experience and Market Evaluation

MASFE is a solo submission, so credibility comes from systems execution rather than headcount. Emil Wes Lambert is completing a BSc in Aerospace Engineering at TU Delft with coursework in spacecraft systems engineering, AOCS, mission analysis, and embedded flight software. Two experiences are especially relevant. First, the TU Delft Design Synthesis Exercise required full-mission requirements tracing, subsystem trade studies, and budget closure for a Venus mission concept. Second, two years in AeroDelft as Partnerships and Business Manager built experience in stakeholder alignment, commercial framing, and turning technically ambitious concepts into credible development roadmaps. That combination maps directly onto MASFE's mix of SWAP realism and market positioning.

The commercial case is a focused greenfield entry rather than a vague platform claim. A reasonable market framing is a \$12B precision-agriculture TAM, a roughly \$2.5B satellite-analytics SAM, and a sub-\$50M near-term SOM for sub-day adaptive pathogen alerting. MASFE's pricing target of \$3–5/ha/year supports break-even at roughly 500,000 ha with a small hosted constellation, while the software layer remains the defensible asset because the orchestration engine can be relicensed across missions.

That generalization story matters for the judges' business-model criterion. Agriculture is the primary entry market, but the same scheduling architecture can be retuned for wildfire fuel-moisture monitoring, flood-front tracking, or urban heat-island response with new science products and the same compute-and-downlink policy core. In other words, MASFE is not only a farm-monitoring product; it is an onboard resource-allocation engine with agriculture as its strongest first proof point.

5 Expected Next Steps

MASFE is currently a TRL 3 concept: the scheduling logic, fusion equation, and SWAP closure are demonstrated in software, but the payload has not yet been exercised on target hardware. The next milestone is a TRL 4 hardware-in-the-loop demonstration on a LEON4FT-class evaluation platform using real EarthData scenes. After that, the cleanest TRL 5 path is a hosted-payload pilot through Loft Orbital, because it validates the business model and the operations concept at the same time.

If selected as a finalist, the highest-value actions before the June pitch event are straightforward:

1. run the simulation on real MODIS tiles and show the policy decisions live;
2. turn the hosted-payload concept into a first-pass unit-economics model with Loft pricing and farm-platform pricing;

3. validate the alert-latency and pricing assumptions through farmer interviews and one agronomy advisor review.

The submission's main claim is therefore not that MASFE is flight-ready today. It is that the adaptive-sensing architecture is already clear, quantitative, and commercially legible enough to justify the next round of validation.

References

- [1] NASA Earth Science Technology Office, *Space-to-Soil Challenge: Challenge Brief and Guidelines*, 2026. <https://nasa-space-to-soil.org>.
- [2] NASA EOSDIS Land Processes DAAC, *MOD13A1 MODIS/Terra Vegetation Indices 16-Day L3 Global 500 m SIN Grid V061*, 2021. DOI: <https://doi.org/10.5067/MODIS/MOD13A1.061>.
- [3] NASA EOSDIS Land Processes DAAC, *MOD11A1 MODIS/Terra Land Surface Temperature/Emissivity Daily L3 Global 1 km SIN Grid V061*, 2021. DOI: <https://doi.org/10.5067/MODIS/MOD11A1.061>.
- [4] K. Gold, *Spectral Early Detection of Crop Pathogens for Regenerative Agriculture Applications*, NASA Space-to-Soil Challenge Informational Webinar 1, Cornell University Plant Disease Diagnostics Laboratory, February 12, 2026.
- [5] J. Mytkowicz, *AI-Enabled Satellites and On-Orbit Decision-Making: Lessons from Loft Orbital*, NASA Space-to-Soil Challenge Informational Webinar 2, Loft Orbital, March 25, 2026.
- [6] OpenET Consortium, *FARMS API Documentation*, 2025. <https://openet.gitbook.io/docs/additional-resources/farms>.
- [7] JPL Artificial Intelligence Group, *Scheduling and Planning Technology*, 2024. <https://ai.jpl.nasa.gov/public/projects/>.
- [8] California Polytechnic State University, *CubeSat Design Specification Rev. 14.1*, 2022. <https://www.cubesat.org>.

A SWAP Budget and Simulation Results

This appendix provides the detailed supporting budgets referenced in the main paper. Values are traceable to representative component datasheets for LEON4FT-class processing, Kintex UltraScale+ RT acceleration, hosted-payload power storage, and conservative CubeSat-class structural allocations [8].

Power Budget

Table 2: Hosted payload module power budget. Solar generation average 10.5 W BOL at 500 km SSO equivalent.

Component	Mode	Draw (W)	Duty (%)	Avg (W)
Kintex UltraScale+ RT FPGA	Always-on preprocess-ing	1.80	100	1.80
LEON4FT CPU (idle + MDP)	Continuous scheduling	1.40	100	1.40
MOD13A1 EVI computa-tion	Stage 1 screening	0.80	55	0.44
MOD11A1 LST computa-tion	Stage 2 fusion only	0.90	18	0.16
CSC fusion task	Stage 2 fusion only	0.50	18	0.09
Multispectral sensor	Observation windows	1.50	50	0.75
Thermal IR sensor	Stage 2 only	0.70	18	0.13
X-band transmitter	Priority downlink passes	2.00	14	0.28
Bus (OBC, ADCS, comms hw)	Continuous	2.50	100	2.50
Total average (MASFE MDP)				7.55 W
Peak (FUSE_PRIORITY)		9.60		—
Naive raw-dump baseline		9.60	~79	9.60 W

Solar generation (10.5 W) minus MASFE average (7.55 W) = **+2.95 W margin**. All modes **PASS**.

Compute Budget

Table 3: Compute budget. LEON4FT: 600 DMIPS. Kintex UltraScale+ RT: ~1.1 M LUTs.

Task	Processor	DMIPS / LUTs	Latency	Status
Radiometric correction + band ex-tract	FPGA	~45k LUTs	<8 ms/frame	PASS
CSC per-pixel computation	FPGA	~22k LUTs	<7 ms/frame	PASS
MOD13A1 EVI anomaly scoring	LEON4FT	180 DMIPS	~15 ms	PASS
MOD11A1 LST retrieval	LEON4FT	200 DMIPS	~18 ms	PASS
CSC fusion inference	LEON4FT	120 DMIPS	~10 ms	PASS
MDP policy evaluation	LEON4FT	55 DMIPS	<1 ms	PASS
Total CPU (all tasks active)	LEON4FT	555 DMIPS (92.5 %)		PASS

Data Budget

Table 4: Data volume budget. X-band downlink: ~ 50 Mbps peak. Storage: 8 GB flash.

Mode / product	Per pass (MB)	Season total (MB)	vs Raw
Raw LIB imagery (naive baseline)	120.0	14,400	—
Fixed-schedule onboard (no MDP)	2.8	330	−98 %
MASFE Stage 1 only (no alert)	0.7	—	−99 %
MASFE Stage 2 + CSC map	2.8	—	−98 %
MASFE priority alert (5 patches)	~ 10.4	—	−91 %
MASFE MDP season average (simulation)	2.6	313	−98 %

Mass and Volume Budget

Table 5: Hosted payload module mass and volume budget, allocated conservatively within a 6U-class envelope.

Component	Mass (g)	Volume	Notes
LEON4FT CPU board	~ 120	~ 0.3 U	GR-CPCI-LEON4-N2X class board
Kintex UltraScale+ RT FPGA	~ 180	~ 0.4 U	Radiation-tolerant accelerator board
Multispectral sensor head	~ 350	~ 1.0 U	Simera xScape100-class payload
Thermal IR sensor	~ 130	~ 0.3 U	Compact hosted-payload TIR module
X-band / host interface	~ 320	~ 0.4 U	Downlink and bus interface electronics
Battery + power conditioning	~ 420	~ 0.4 U	Local storage and regulation margin
Structure, harness, thermal	~ 380	~ 0.2 U	Payload-only mechanical allocation
Total estimated	$\sim 1,900$ g	~ 3.0 U	Within hosted-payload limits

Simulation Performance Results

B Python Simulation Code Excerpts

This appendix contains the key components of the MASFE Python simulation submitted as Code Submission Component 3. The full source file (`masfe_simulation.py`) is available in the linked GitHub repository. The simulation runs with only `numpy` as a dependency (`pip install numpy`).

Running the simulation produces the three-way comparison output (MASFE MDP vs fixed-schedule onboard vs naive raw-dump baseline) and exports the JSON metrics block used in the paper. Verified output:

- Downlink reduction vs raw: **97.8 %**
- Energy saving vs raw: **22.0 %**
- Science retention: **100 %**

Table 6: Three-way performance comparison: raw dump vs fixed onboard vs MASFE MDP. Results from simulation over a 120-pass growing season (50 field patches, 7 disease events, 38 cloud-obscured passes, benign stress confounders, and sensor noise).

Metric	Raw dump	Fixed onboard	MASFE MDP
Downlink data (MB, season)	14,400	330	313
Active payload energy (Wh)	9.60	7.90	7.48
True pathogen detections	304	307	305
False positives	3	3	3
Priority alerts dispatched	—	—	13
Min battery SOC	—	—	86 %
Downlink reduction vs raw	—	−97.7 %	−97.8 %
Energy saving vs raw	—	−17.7 %	−22.0 %
Science retention	100 %	100 %	100 %
False-positive rate	1.0 %	1.0 %	1.0 %
Alert latency	1–3 days	<90 min	<90 min

- False-positive rate: **1.0 %** with cloud and benign-stress confounders
- Peak power: **9.6 W** $\leq$ 14 W budget PASS
- Minimum battery SOC: **86 %** $>$ 15 % floor PASS

CSC Fusion Function

```

1 def compute_csc(evi_t, lst_t, evi_base, lst_base):
2     """
3     Crop Stress Composite -- per-patch anomaly index.
4     Inputs:
5         evi_t      : observed EVI per patch (MOD13A1)
6         lst_t      : observed LST per patch in K (MOD11A1)
7         evi_base   : per-patch healthy-season EVI baseline
8         lst_base   : per-patch healthy-season LST baseline
9     Returns:
10        csc : array in [0, 1], 0=healthy, 1=severe stress
11    """
12    sigma_e = 0.042    # EVI noise + phenology (dimensionless)
13    sigma_l = 1.20    # LST noise (K)
14
15    evi_drop = np.maximum(0.0, (evi_base - evi_t) / sigma_e) / 5.0
16    lst_rise = np.maximum(0.0, (lst_t - lst_base) / sigma_l) / 4.0
17
18    csc = 0.55 * np.clip(evi_drop, 0, 1) \
19          + 0.45 * np.clip(lst_rise, 0, 1)
20    return np.clip(csc, 0, 1)

```

Listing 1: Onboard Crop Stress Composite (CSC) fusion from MOD13A1 EVI and MOD11A1 LST. Equations correspond to Eq. (1) in the main paper.

MDP Policy

```

1 class MASFEPolicy:
2     """

```



```

3  Two-stage adaptive MDP scheduler.
4  Stage 1 (MOD13): cheap EVI screening every pass.
5  Stage 2 (FUSE): triggered by EVI anomaly or periodic
6                    exploration (every explore_n steps).
7  Degradation:
8      SOC > 35% : full two-stage MDP (NOMINAL)
9      15-35%   : Stage 1 (MOD13) only (CONSERVE)
10     < 15%    : SKIP -- preserve battery (CRITICAL)
11  """
12  def __init__(self, evi_fuse_thr=0.18, csc_alert_thr=0.55,
13                explore_n=12, batt_crit=0.15, batt_cons=0.34):
14      self.evi_fuse_thr = evi_fuse_thr
15      self.csc_alert_thr = csc_alert_thr
16      self.explore_n = explore_n
17      self.batt_crit = batt_crit
18      self.batt_cons = batt_cons
19
20  def act(self, state):
21      soc = state.soc
22      max_anom = state.evi_anom.max()
23      max_csc = state.csc.max()
24
25      # CRITICAL: preserve battery
26      if soc < self.batt_crit:
27          return 'SKIP'
28
29      # CONSERVE: Stage 1 screening only
30      if soc < self.batt_cons:
31          return 'MOD13'
32
33      # Confirmed alert + downlink window -> priority TX
34      if max_csc >= self.csc_alert_thr and state.downlink:
35          return 'FUSE_PRIORITY'
36
37      # EVI anomaly triggers Stage 2 confirmation
38      if max_anom >= self.evi_fuse_thr:
39          return 'FUSE'
40
41      # High CSC but no downlink yet -> re-confirm, buffer
42      if max_csc >= self.csc_alert_thr:
43          return 'FUSE'
44
45      # Periodic exploration (MDP discovery)
46      if state.steps_since_fuse >= self.explore_n:
47          return 'FUSE'
48
49      # Default: cheap EVI screening
50      return 'MOD13'

```

Listing 2: MASFE two-stage MDP scheduling policy. Selects action from {SKIP, MOD13, FUSE, FUSE_PRIORITY} based on battery SOC, EVI anomaly score, CSC level, and downlink window availability.

ISA Atmosphere Helper (TU Delft Template Reference)

The ISA atmosphere model (shown in Appendix A of the TU Delft template) was used to verify the 500 km orbit altitude for solar flux and eclipse fraction calculations used in the power budget:

```

1 import math
2 g0, R = 9.80665, 287.0
3 layer1 = [0, 288.15, 101325.0]
4 alt = [0, 11000, 20000, 32000, 47000, 51000, 71000, 86000]
5 a = [-.0065, 0, .0010, .0028, 0, -.0028, -.0020]
6
7 def atmosphere(h):
8     for i in range(len(alt) - 1):
9         if h >= alt[i]:
10             layer0 = layer1[:]
11             layer1[0] = min(h, alt[i + 1])
12             if a[i] != 0:
13                 layer1[1] = layer0[1] + a[i] * (layer1[0] - layer0[0])
14                 layer1[2] = layer0[2] * (layer1[1] / layer0[1]) \
15                     ** (-g0 / (a[i] * R))
16             else:
17                 layer1[2] = layer0[2] * math.exp(
18                     (-g0 / (R * layer1[1])) * (layer1[0] - layer0[0]))
19             T, rho, P = layer1[1], layer1[2]/(R*layer1[1]), layer1[2]
20             return [T, rho, P, math.sqrt(1.4 * R * T)]

```

Listing 3: ISA Calculator used for orbit power budget verification. Adapted from TU Delft report template Appendix A.